第 06 章

LangGraph 节点、条件边与中断恢复

用真实 StateGraph 和固定资料观察状态合并、路径选择、人工中断与同线程恢复

这一章完成后，你能够

- 把 State、Node、Edge、Reducer 和 Checkpointer 映射到可运行代码。
- 跟踪有资料与缺资料两条路径，理解条件边和节点返回值的区别。
- 执行批准、拒绝和缺资料案例，解释 interrupt 与 Command(resume=...)。
- 将独立教学图与生产 create_agent 图对应，准确说明内存实验的边界。

学习前提

第05章的消息、工具、Command 与结构化结果；Python TypedDict 与 Annotated 基础；默认环境 LangGraph 1.2.12

代码基准：8aa0ba5 + 本轮名称与分页修改。Python 3.12；LangChain 1.4.3；LangGraph 1.2.12。

正文采用场景、名词、步骤、源码、实验和练习的教学顺序。截图注明数据来源；教学简化与生产实现分别标记。

本章目录

6.1 为什么单独写一个不调用模型的图 3

6.2 五个核心名词先对应到对象 4

6.3 局部更新与 reducer：空列表为何没清空 5

6.4 节点返回与条件路由 6

6.5 编译图：结构变成可执行对象 7

6.6 interrupt：暂停位置与恢复输入 8

6.7 恢复会重新进入节点：副作用要谨慎 9

6.8 完整运行三条路径并解释输出 10

6.9 映射回生产 Agent 与官方接口 12

6.10 综合练习与参考思路 13

6.11 源码阅读清单与下一步 14

6.1 为什么单独写一个不调用模型的图

上一章的 Agent 路径由模型和工具结果共同决定。如果你同时学习模型行为与图状态，很容易把模型决定和框架机制混在一起。本章使用固定 TCP 资料，不接外部模型和数据库，只观察“检索→判断依据→生成草稿→等待审核”的控制过程。

本章实验不是生产功能

examples/learning/langgraph_review.py 是独立教学图。产品辅导图由 create_agent 构建；产品没有因为这个实验而新增人工审核按钮或手写多节点审批流程。InMemorySaver 只在当前进程保存，不证明重启后持久恢复。



图 6-1 教学图概览。draft 与 missing 是互斥分支，下一张图展开实际条件。



图 6-2 真正的路径分支。缺资料案例不会先起草，也不会进入审核。

把输入 topic 设为 TCP，有资料，图会暂停等待审核；设为量子纠缠，固定资料为空，图直接返回证据不足。这个对照能验证路由与终止行为，不能用于判断模型能否搜索量子知识，因为实验根本没有接入模型或真实检索。

6.2 五个核心名词先对应到对象

名词	含义	教学代码
State / 状态	图共享字段及字段更新规则	ReviewState
Node / 节点	读取状态并返回局部更新的函数	retrieve、draft、review、missing
Edge / 边	固定跳转或按状态选择去向	add_edge；add_conditional_edges
Reducer / 合并函数	旧值与新增更新怎样合成	trace 的 operator.add
Checkpointer / 保存器	保存执行状态供同线程继续	InMemorySaver
Compiled graph / 可执行图	编译后提供 invoke、get_state 等方法	builder.compile(...)

表 6-1 术语必须对应具体对象，而不是把每个节点都解释为一个 Agent

节点可以调用模型，也可以只运行普通 Python。本章 retrieve 用条件表达式返回固定字符串，draft 直接复制证据。仍然使用真实 LangGraph 执行，因为图机制负责调度、状态合并、保存与恢复，和节点内部是否有模型是不同问题。

源码节选：状态类型与检索节点

定位：examples/learning/langgraph_review.py，第 8-30 行。

```python
from typing import Annotated, Literal, TypedDict
from uuid import uuid4

from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command, interrupt


class ReviewState(TypedDict, total=False):
    topic: str
    evidence: str
    answer: str
    status: str
    trace: Annotated[list[str], operator.add]


def retrieve(state: ReviewState) -> ReviewState:
    evidence = "TCP 用三次握手建立连接。" if state["topic"] == "TCP" else ""
    return {"evidence": evidence, "trace": ["retrieve"]}


def route(state: ReviewState) -> Literal["draft", "missing"]:
    return "draft" if state["evidence"] else "missing"
```

`TypedDict` 描述字典键的类型；`total=False` 允许某些字段在流程早期还不存在。它主要服务类型表达，不会自动给每个键设置默认值。

`Annotated[list[str], operator.add]` 将 trace 的类型与合并规则一起声明。retrieve 返回 evidence 和一个新 trace 项，框架把更新应用到原状态。route 返回节点名称，负责选路而不是产生新证据。

输入只需要 topic 和 trace；evidence、answer、status 会在后续节点逐步出现。如果某个节点直接读取未生成的键，仍会发生普通 Python KeyError。画图时除了连接节点，还要记录每个节点需要哪些前置字段，以及谁负责生成它们。

6.3 局部更新与 reducer：空列表为何没清空

假设旧 trace 是 [retrieve]，draft 返回 [draft]。字段有 operator.add 时，新状态为 [retrieve, draft]；如果没有 reducer，默认更新会覆盖旧值。节点通常返回它修改的局部字段，不必复制整个字典。只有明确理解每个字段更新方式，才能推断最后结果。

更新方式	旧值与更新	合并结果
默认覆盖	count=0；更新 count=1	count=1
operator.add	trace=[start]；更新 [append]	trace=[start, append]
空更新仍合并	trace=[start, append]；更新 []	原有项目保留
Overwrite 绕过 reducer	trace=[start, append]；更新 Overwrite([])	trace=[]

表 6-2 空值、追加与显式覆盖的行为不同

完整可运行实验：亲自比较空更新与显式清空

```python
import operator
from typing import Annotated, TypedDict
from langgraph.graph import START, END, StateGraph
from langgraph.types import Overwrite

class State(TypedDict):
    trace: Annotated[list[str], operator.add]
    count: int

def append(state: State):
    return {"trace": ["append"], "count": state["count"] + 1}

def empty(state: State):
    return {"trace": []}

def clear(state: State):
    return {"trace": Overwrite([])}

for reset in [empty, clear]:
```

代码续

```
    builder = StateGraph(State)
    builder.add_node("append", append)
    builder.add_node("reset", reset)
    builder.add_edge(START, "append")
    builder.add_edge("append", "reset")
    builder.add_edge("reset", END)
    result = builder.compile().invoke({"trace": ["start"], "count": 0})
    print(reset.__name__, result)
```

两个图都先执行 append，使 count 从 0 变为 1。empty 返回空列表，仍按追加规则合并；clear 返回 Overwrite，明确绕过合并规则。

`builder.compile().invoke(...)` 在这里没有保存器，只运行一次。实验不测试中断或持久性；它只验证 reducer 的真实字段更新规则。

结果解释：empty 的 trace 保留 ['start', 'append']；clear 的 trace 变为 []；两个结果的 count 都为 1。

生产代码下一轮重建消息和本轮证据时也需要这个区别。若把 evidence={} 当作“清空”，但该字段使用字典合并，旧引用可能仍留下。第07章会查看 TutorService 如何用 Overwrite 明确隔离本轮状态。不要为了方便把所有 reducer 改成覆盖，消息和轨迹仍有累积需求。

6.4 节点返回与条件路由

源码节选：有依据起草、审核和缺资料终止

定位：examples/learning/langgraph_review.py，第 33-50 行。

```
def draft(state: ReviewState) -> ReviewState:
    return {"answer": state["evidence"], "trace": ["draft"]}


def review(state: ReviewState) -> ReviewState:
    # 恢复时整个节点会再次执行，因此 interrupt 前不进行写库等副作用。
    approved = interrupt({"answer": state["answer"], "evidence": state["evidence"]})
    if not isinstance(approved, bool):
        raise ValueError("审核结果必须是 bool")
    return {"status": "accepted" if approved else "rejected", "trace": ["review"]}


def missing(state: ReviewState) -> ReviewState:
    return {
        "answer": "当前固定资料没有依据。",
        "status": "insufficient_evidence",
        "trace": ["missing"],
    }
```

`draft(state)` 读取 evidence，返回 answer 与 trace；它不选择下一条边。`review(state)` 返回最终 status；批准和拒绝都形成终态。

`missing(state)` 返回一段固定的证据不足说明。它使用同一个状态类型，但通过不同边结束。资料内容在本实验只支持 TCP，不应让缺资料输入落入草稿节点。

函数	输入条件与返回	常见误解
retrieve	topic→evidence 与 trace 更新	名称叫检索，不表示连接了真实向量库
route	evidence→draft 或 missing	返回路由名称，不直接写 answer
draft	evidence→answer	此处复制固定文本，不是 LLM 生成
review	审核 resume 值→accepted / rejected	拒绝也是终态，不是自动再次生成
missing	无依据→insufficient_evidence	缺资料分支不会调用 interrupt

表 6-3 函数名、输入和返回逐一解释

这里使用普通条件边，因为路线只由一个状态条件决定。若既要更新字段又要指定跳转，可以学习 Command(update=..., goto=...)；生产工具的 Command 用于更新消息和证据，不应直接套成本章 resume 的用法。同一个类在不同位置承担不同命令含义，必须看参数和调用位置。

条件边的映射要和节点名字一致

route 返回 draft 或 missing，builder 注册了同名节点。拼错名字或遗漏连边会导致编译或执行问题。先画清成功和缺资料两条完整路径，再补异常路径，比边写边猜更可靠。

6.5 编译图：结构变成可执行对象

源码节选：注册节点、边与保存器

定位：examples/learning/langgraph_review.py，第 53-69 行。

```
53  def run_demo() -> dict[str, object]:
54      builder = StateGraph(ReviewState)
55      for name, node in [
56          ("retrieve", retrieve),
57          ("draft", draft),
58          ("review", review),
59          ("missing", missing),
60      ]:
61          builder.add_node(name, node)
62      builder.add_edge(START, "retrieve")
63      builder.add_conditional_edges("retrieve", route)
64      builder.add_edge("draft", "review")
65      builder.add_edge("review", END)
66      builder.add_edge("missing", END)
67      graph = builder.compile(checkpointer=InMemorySaver())
68      results = []
69      for approved in [True, False]:
```

`StateGraph(ReviewState)` 声明图状态；add_node 注册函数；START 与 END 是框架的入口和终止标记。

`add_conditional_edges("retrieve", route)` 在检索后调用路由函数选择去向。draft 固定到 review，missing 固定到 END。compile 返回可执行图，并装配 InMemorySaver。

编译用于把节点、边和运行参数组织成可执行图，也会做结构检查。它不会把 Python 函数编译为机器码，更不会因为某节点名叫 draft 就自动添加模型。图编译后仍需 invoke 或 ainvoke 传入状态。先构图，后执行，是两个明确阶段。

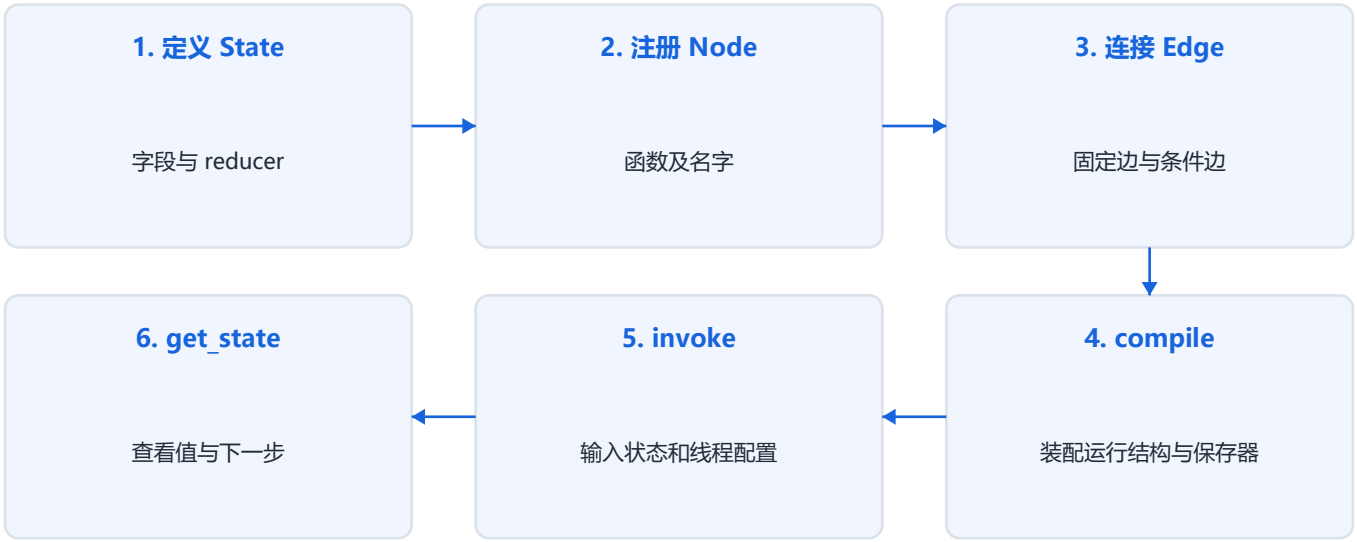


图 6-3 从构图到运行的顺序；保存器在 compile 时绑定，线程身份在执行配置中提供。

如果后来添加“审核拒绝→重写草稿”的回路，需要明确退出条件和预算。循环图本身不会保证最终结束。当前教学例子 intentionally 使用一次审核终态，方便你观察 accepted 与 rejected 的区别；生产 Agent 的循环则由结构化输出和预算控制收束。

6.6 interrupt：暂停位置与恢复输入

review 节点调用 interrupt，向调用者暴露当前 answer 和 evidence，并等待一个恢复值。在本实验中该值必须是 bool。第一次 invoke 返回带 __interrupt__ 的结果；get_state(config).next 表示下一步等待 review。暂停不是一个普通业务失败，也不代表回复已经被审核接受。

对象或参数	作用	本例中的值
interrupt(payload)	暂停并提供需要人工查看的数据	answer 与 evidence 字典
configurable.thread_id	保存器中区分执行线程	每个案例一个 uuid4 字符串
__interrupt__	首轮输出中的中断信息	提示图在等待外部决定
Command(resume=True)	给中断位置提供恢复值	approved=True → accepted
Command(resume=False)	以不同决定继续同一线程	approved=False → rejected
get_state(config).next	查看待执行的节点	("review",)

表 6-4 线程身份、暂停信号和人工决定分别对应什么

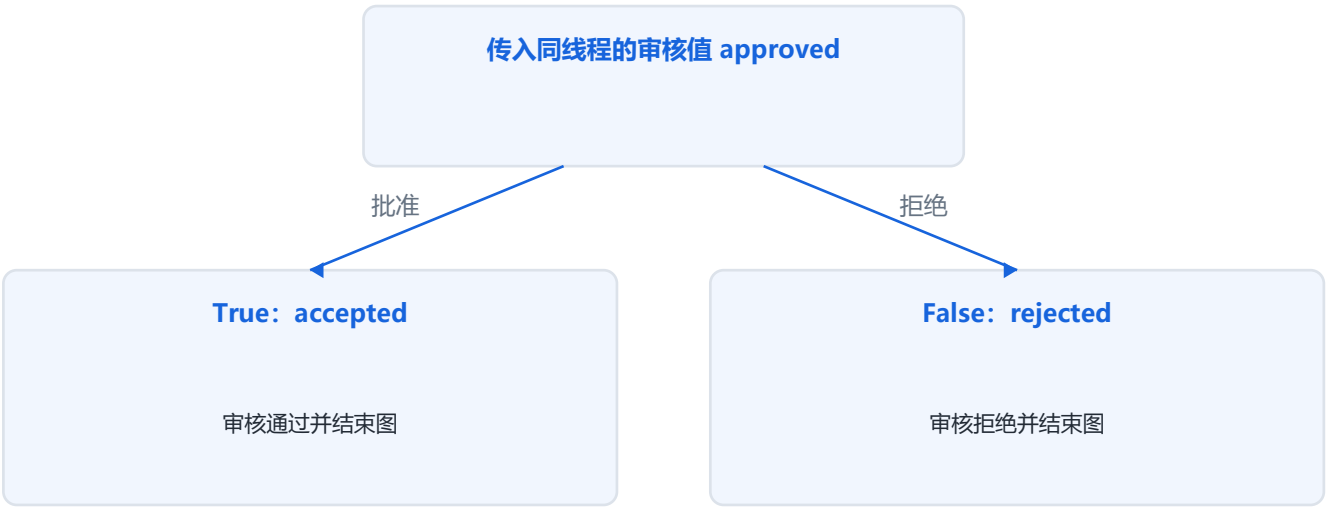


图 6-4 审核决定分支。两条路径都结束，不表示拒绝后自动重写。

恢复必须沿用同一个图、保存器和 thread_id。换一个 ID 就换了保存器中的执行线程，新的线程没有旧中断状态。相反，把多个独立用户错误地放到同一个 ID，会混淆状态身份。生产项目用固定会话线程与业务轮次映射管理这个关系，而不是只凭聊天问题文字猜测线程。

6.7 恢复会重新进入节点：副作用要谨慎

源码节选：同线程暂停与恢复断言

定位：examples/learning/langgraph_review.py，第 70-89 行。

```
        config = {"configurable": {"thread_id": str(uuid4())}}
        paused = graph.invoke({"topic": "TCP", "trace": []}, config)
        assert paused["__interrupt__"]
        assert graph.get_state(config).next == ("review",)
        # 恢复需同一 thread_id、兼容图结构及可访问原检查点的保存后端。
        resumed = graph.invoke(Command(resume=approved), config)
        expected = "accepted" if approved else "rejected"
        assert resumed["status"] == expected
        assert resumed["trace"] == ["retrieve", "draft", "review"]
        results.append(
            {
                "decision": approved,
                "status": resumed["status"],
                "trace": resumed["trace"],
                "paused_at": "review",
            }
        )
    no_evidence = graph.invoke(
        {"topic": "量子纠缠", "trace": []},
        {"configurable": {"thread_id": str(uuid4())}},
```

首次 invoke 传入 topic 与初始 trace；恢复调用传入 Command，不重新提交一份 topic 状态。config 保持相同，才能找到等待的审核位置。

断言除了检查 status，还检查 trace 为 retrieve、draft、review。这能证明最后走过哪些节点，但不能证明生产业务事务或真实模型质量。

从中断恢复时，review 节点会从函数开头重新执行，interrupt 返回本次恢复提供的值。因此不要在 interrupt 之前随手写入“审核通过”或发送通知。那些代码可能在第一次暂停和恢复时重复执行，必须设计幂等保护或将副作用放到审核结果明确之后的受控阶段。

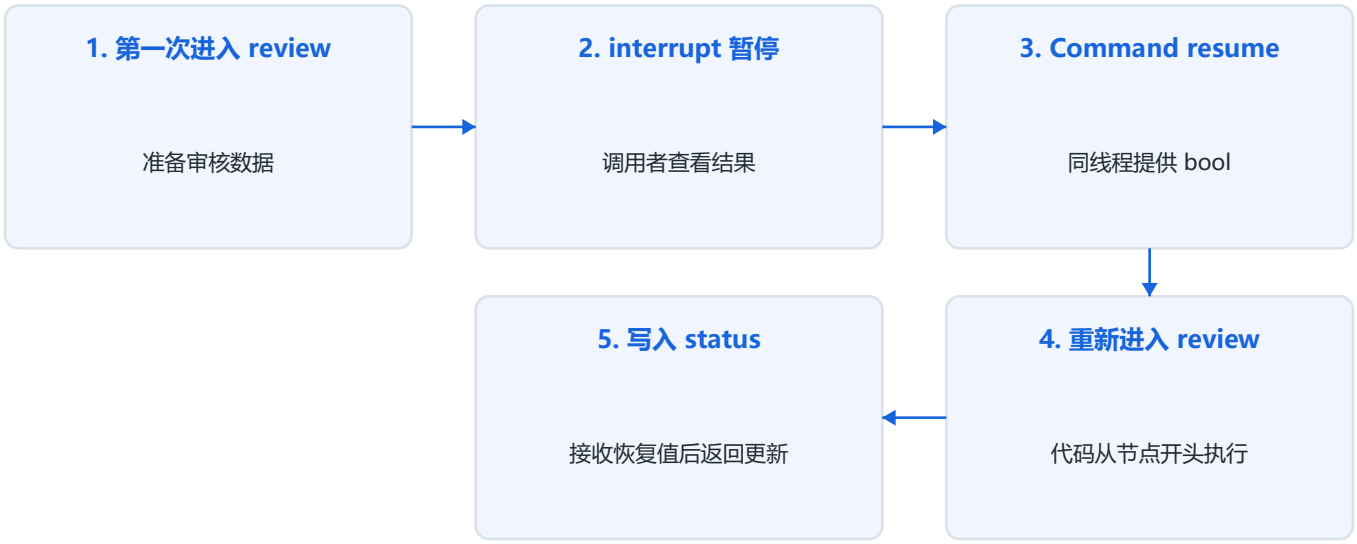


图 6-5 恢复语义说明。图恢复和函数从 Python 行号原地继续的直觉不同。

本例为什么安全简单

interrupt 之前只有构造字典和读状态，没有业务数据库写入。每次独立审核案例使用新的线程。不要把 InMemorySaver 换成 PostgreSQL 后就宣称整个业务具备 exactly once，正式提交仍需要第07章的幂等键、锁、事务和接受指针。

若要扩展这个例子，可在 review 返回之后单独增加 publish 节点，再为发布动作设计业务幂等 ID。但这只是设计练习，不能在尚无正式需求与验证时把它描述成产品已有能力。本章以最小实验说明框架行为，生产一致性另有约束。

6.8 完整运行三条路径并解释输出

独立实验文件已经包含完整 imports、节点定义和构图。下面的完整调用代码从项目根目录加载它，并打印关键结果。使用默认 apps/api/.venv/Scripts/python.exe 运行；不需要数据库、Embedding 或 API Key。runpy 不导入为项目包，也不会执行该文件的 __main__ 输出分支。

完整可运行实验：调用完整教学图并检查状态

```python
import runpy
from pathlib import Path

module = runpy.run_path(str(Path.cwd() / "examples/learning/langgraph_review.py"))
result = module["run_demo"]()
for case in result["review_cases"]:
    print(case["decision"], case["status"], case["trace"])
missing = result["missing_case"]
print("missing:", missing["status"], missing["trace"])
```

`run_demo()` 内部实际执行批准、拒绝和缺资料三种案例，不是预先拼接输出。批准和拒绝都检查了首次中断位置及同线程恢复结果。

固定资料使成功路径可重复。trace 验证路径，status 验证终态，missing_case 说明资料不足时图不会进入审核。

结果解释：输出 True accepted、False rejected；两者 trace 为 ['retrieve', 'draft', 'review']。missing 为 insufficient_evidence，trace 为 ['retrieve', 'missing']。

输入与决定	最终状态	本次制作实际运行结果
TCP + True	accepted	先暂停 review，再恢复；完整 trace 三节点
TCP + False	rejected	同样先暂停，拒绝值决定终态
量子纠缠	insufficient_evidence	retrieve→missing，无审核中断

表 6-5 真实框架加固定资料的三条路径，不是供应商评测

建议先原样运行，再修改一个变量：把 TCP 改成别的主题，观察证据不足；把恢复值改成字符串，观察 bool 校验；尝试新 thread_id，观察找不到旧暂停状态的结果。每次修改应说清你预计哪个字段或路径变化，避免只看输出是否“像成功”。

6.9 映射回生产 Agent 与官方接口

源码节选：生产 Agent 的图入口与策略

定位：apps/api/study_agent/agents/tutor/agent.py，第 24-49 行。

```
    queries: LearningQueries,
    retrieval: LearningMaterialRetrieval,
    budget: RunBudget,
    checkpointer: BaseCheckpointSaver[Any],
) -> CompiledStateGraph[Any, Any, Any, Any]:
    repairs = 0

    def repair_once(error: Exception) -> str:
        nonlocal repairs
        repairs += 1
        if repairs > 1:
            raise DomainError("TUTOR_INVALID_OUTPUT", "结构化输出修复失败", status_code=503)
        return "输出格式不符合 TutorAnswerDraft。请修复一次并保留真实证据引用。"

    return create_agent(
        model=model,
        tools=make_tutor_tools(queries, retrieval),
        system_prompt=SYSTEM_PROMPT,
        response_format=ToolStrategy(TutorAnswerDraft, handle_errors=repair_once),
        state_schema=TutorState,
        context_schema=TutorContext,
        checkpointer=checkpointer,
        middleware=cast(
            Any,
            [
                TutorBudgetMiddleware(budget),
```

`create_tutor_agent` 返回 CompiledStateGraph。生产图由 LangChain create_agent 构建，state_schema、context_schema、工具、输出策略和保存器作为参数装配。

生产图执行模型工具循环，没有使用本章 review 节点。ToolStrategy 的修复限制与业务最终校验属于输出可信性控制；人工审核中断属于独立教学练习。

教学图	生产辅导	相同与不同
ReviewState	TutorState	都声明图状态；生产有消息和证据合并规则
固定 retrieve	真实资料检索工具	实验控制输入；生产还核对当前资料和知识库
draft 复制原文	模型产出结构化草稿	图机制相通，生成与可信性要求不同
InMemorySaver	AsyncPostgresSaver	保存器契约相通；重启持久性不同
interrupt / resume	业务失败恢复与接受指针	人工中断不等于失败后的业务草稿接受

表 6-6 先建立图机制，再理解生产约束，避免把示例功能投射进产品



图 6-6 真实辅导历史页面用于观察生产会话入口。它不是本章独立人工审核图的界面。

历史项目截图 / 2026-10-07；用于说明原有界面流程。来自原教材对应页面；截图时的业务数量不代表当前状态

框架接口已对照官方 Graph API 与 Interrupts，并以锁定 LangGraph 1.2.12 执行。阅读在线文档时可能看到更新版本的新写法；先确认教材版本，再决定是否迁移。不要只复制最新示例而忽略本地依赖，尤其是状态类型、保存器和恢复配置。

- 官方 Graph API：https://docs.langchain.com/oss/python/langgraph/graph-api
- 官方 Interrupts：https://docs.langchain.com/oss/python/langgraph/interrupts
- 本章关于字段合并、中断、恢复的结论以项目实验实际执行为主要依据。

6.10 综合练习与参考思路

练习 1：节点和路由函数的返回有什么差别？

参考思路：节点返回字段更新；条件路由返回下一节点名称。本例 route 不写 answer，draft 才写 answer。

练习 2：为什么返回 trace=[] 没有清空旧轨迹?

参考思路：trace 使用追加 reducer，旧列表与空列表合并仍是旧列表。要显式清空可用 Overwrite，但应该只在需要重建的场景使用。

练习 3：为什么 interrupt 前不写入正式结果?

参考思路：恢复重新执行整个中断节点，中断前代码可能重复运行。正式副作用需设计幂等与业务接受条件。

练习 4：本实验能证明服务重启后恢复吗?

参考思路：不能。InMemorySaver 的状态只在当前进程。生产 PostgreSQL 保存器的持久性还要结合业务事务、接受指针与集成测试审阅。

6.11 源码阅读清单与下一步

文件或入口	关注的函数 / 类	阅读时回答的问题
examples/learning/langgraph_review.py	ReviewState / retrieve / route / review / run_demo	节点、分支和同线程恢复怎样组合
apps/api/study_agent/agents/tutor/state.py	TutorState / merge_evidence / merge_flags	生产状态的合并规则是什么
apps/api/study_agent/agents/tutor/agent.py	create_tutor_agent	生产图装配哪些工具与策略
apps/api/study_agent/agents/tutor/tools.py	Command / ToolMessage	工具更新和恢复命令有何差异
apps/api/study_agent/application/tutoring.py	TutorService.submit	为什么不能盲目恢复最新检查点

下一章：第07章把框架图状态与正式业务结果连接起来，分析检查点、幂等键、锁、事务和失败草稿隔离。

主要参考：本地飞书 LangChain、Agent、RAG 与 LangGraph 课程的教学组织方式，以及当前项目源码。具体框架接口以锁定环境与官方文档为准。